

단위 변환 CLI 프로포절

2026. 10. 09.

목차

1. 요약
 2. 배경과 문제
 3. 사전 조사
 4. 목표와 범위
 5. 마일스톤 계획
 6. 위험과 대응
- 참고 출처

문서의 목적. 길이·무게·온도 단위를 변환하는 Python 명령줄 도구를 표준 라이브러리만으로 만드는 계획을 설명하고, 목표·범위와 두 마일스톤(M1 변환 핵심, M2 명령줄 인터페이스)의 승인을 받는다.

1 요약

저장소는 빈 초기 커밋 하나뿐이다[D1]. 이 목표는 `python convert.py 100 km mi`처럼 값·원래 단위·바꿀 단위를 받아 `62.13711922 mi`를 출력하는 도구를 만든다.

- **방식.** 길이와 무게는 기준 단위(m, kg)에 대한 배수 표 하나로 변환하고, 온도는 켈빈을 거치는 식으로 따로 변환한다. 이것이 Pint와 GNU Units가 쓰는 구분이다[R4, R5].
- **근거 수치.** 환산 계수는 NIST와 1959년 국제 야드·파운드 협정의 정확값을 쓴다($1\text{ in} = 0.0254\text{ m}$, $1\text{ lb} = 0.45359237\text{ kg}$)[R1, R3].
- **제약.** 외부 패키지 없이 Python 3.11 표준 라이브러리(`argparse`, `unittest`)만 쓴다. 파일은 `convert.py`와 시험 파일 둘이다.
- **계획.** M1에서 변환 함수와 시험을, M2에서 명령줄 해석과 오류 처리를 만든다.

2 배경과 문제

지금 저장소에는 코드가 없다[D1]. 실행 환경은 Python 3.11.16이다[D2]. 만들 것은 작지만, 세 종류의 단위가 같은 방식으로 변환되지 않는다는 점이 설계의 핵심이다.

길이와 무게는 곱셈만으로 변환된다. 1 km는 1000 m이고 0 km는 0 m이다. 온도는 눈금의 0점이 서로 다르므로 곱셈에 더해 덧셈이 필요하다. 0 °C는 0 °F가 아니라 32 °F다[R1]. 배수 표 하나로 온도까지 처리하면 이 덧셈이 빠져 틀린 값이 나온다. 표 1은 이 차이를 정리한 것이다.

표 1. 단위 종류별 변환 성질. “선형”은 $y = a \cdot x$, “아핀”은 $y = a \cdot x + b$ 꼴의 변환을 뜻한다. [R1, R4]

종류	지원할 단위	기준 단위	변환 성질	예
길이	mm, cm, m, km, in, ft, yd, mi	m	선형	1 mi = 1609.344 m
무게	mg, g, kg, t, oz, lb	kg	선형	1 lb = 0.45359237 kg
온도	C, F, K	K	아핀	0 °C = 32 °F = 273.15 K

사용자가 잘못 넣는 입력도 문제다. 없는 단위(km → xx), 종류가 다른 단위(km → kg), 숫자가 아닌 값, 절대영도(0 K) 아래의 온도는 결과 대신 이유가 담긴 오류로 끝나야 한다.

3 사전 조사

3.1 기존 도구

단위 변환에는 이미 잘 알려진 도구가 있다. 표 2는 이 프로젝트가 그대로 쓸 수 있는지와 무엇을 배울 수 있는지를 비교한다.

표 2. 기존 방식 비교. [R4, R5]

방식	내용	이 프로젝트에 쓸 수 있나	배울 점
Pint (Python 라이브러리)	단위 등록부와 수량 객체. 온도를 “곱셈이 아닌 단위”로 따로 다루고, 온도 차이는 별도의 delta 단위로 둔다.	아니다. 외부 패키지라 제약에 어긋난다.	온도는 오프셋 변환으로 분리한다. 온도에 곱셈을 섞으면 오류로 막는다.
GNU Units (명령줄 프로그램)	수천 개 단위 정의 파일. 절대 온도는 tempF(45) 같은 함수 문법, 온도 차는 degF 같은 일반 단위로 구분한다.	아니다. 별도 설치가 필요한 외부 프로그램이다.	절대 온도와 온도 차를 구분해야 한다. 이번 범위는 절대 온도만이다.
직접 구현 (배수 표 + 온도 식)	단위 → 기준 단위 배수의 사전 두 개와 온도 변환 식.	그렇다. 표준 라이브러리만으로 수십 줄이면 된다.	—

3.2 환산 계수

계수는 손으로 계산하지 않고 정의값을 그대로 쓴다. 표 3의 값은 모두 정의상 정확한 값이다. NIST 표는 파운드를 7자리로 반올림해 싣고 있으므로(0.4535924)[R2], 파운드는 1959년 국제 야드·파운드 협정의 정의값을 쓴다[R3]. 온스는 파운드의 1/16로 정의되므로 계산으로 얻는다.

표 3. 사용할 정확 환산 계수. 미터·킬로그램 접두어(mm, km, mg 등)는 10의 거듭제곱이므로 생략한다. [R1, R2, R3]

단위	기준 단위로	출처
inch (in)	0.0254 m	NIST SP 811 B.8, 정확값 표시[R1]
foot (ft)	0.3048 m	NIST SP 811 B.8, 정확값 표시[R1]
yard (yd)	0.9144 m	NIST SP 811 B.8, 정확값 표시[R1]
mile (mi)	1609.344 m	NIST SP 811 B.8, 정확값 표시[R1]
pound (lb)	0.45359237 kg	1959년 국제 협정 정의[R3]
ounce (oz)	0.028349523125 kg	1 lb ÷ 16
°F → °C	$(t - 32) \div 1.8$	NIST SP 811 B.8, 상수 정확값[R1]
°C → K	$t + 273.15$	NIST SP 811 B.8[R1]

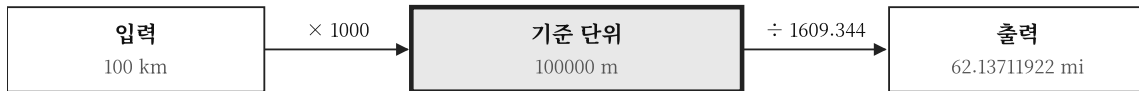
3.3 명령줄 해석

표준 라이브러리의 `argparse`로 위치 인자 세 개(값, 원래 단위, 바꿀 단위)를 받는다. 주의할 점은 음수다. `argparse`는 -로 시작하는 인자를 옵션으로 볼 수 있는데, 음수처럼 보이는 인자는 위치 인자로 받아 준다[R6]. 다만 Python 3.14 이전에는 이 “음수처럼 보이는” 판별이 -40, -1.5 같은 단순한 꼴만 인정하고, -1e3 같은 지수 표기는 3.14에서 추가되었다[R6]. 3.11에서 지수 표기 음수는 --를 앞에 붙여 넘겨야 한다[R6].

3.4 이 프로젝트가 쓸 방식

그림 1은 위 조사를 합친 변환 경로다. 길이와 무게는 기준 단위로 곱했다가 나누고, 온도는 켈빈으로 바꿨다가 다시 바꾼다. 어느 경우든 단위 n 개에 대해 기준 단위와의 관계 n 개만 적으면 되므로, 모든 단위 쌍(n^2 개)을 적지 않아도 된다.

(a) 길이·무게 (선형)



(b) 온도 (아핀)

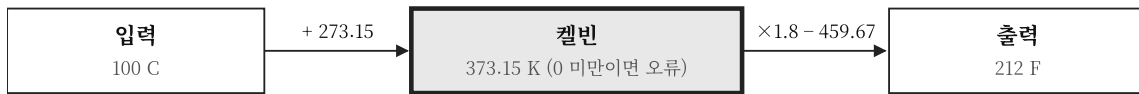


그림 1. 두 가지 변환 경로. 굵은 테두리가 모든 단위가 거쳐 가는 중간 값이다. 길이·무게(a)는 곱셈과 나눗셈만, 온도 (b)는 덧셈이 섞인 식을 쓴다. 숫자는 실제 예시값이다.

4 목표와 범위

목표: 표준 라이브러리만 쓰는 `convert.py` 하나로 길이·무게·온도의 절대값을 정확한 정의 계수로 변환한다. 성공 기준은 표 4와 같다.

표 4. 성공 기준.

기준	측정	목표
정확성	표 3의 정의값과 기준점($-40\text{ }^{\circ}\text{C} = -40\text{ }^{\circ}\text{F}$, $100\text{ }^{\circ}\text{C} = 212\text{ }^{\circ}\text{F}$, $0\text{ }^{\circ}\text{C} = 273.15\text{ K}$) 시험	상대 오차 $\leq 1\text{e-}12$
왕복 일관성	모든 단위 쌍 $A \rightarrow B \rightarrow A$ 변환	상대 오차 $\leq 1\text{e-}12$
오류 처리	없는 단위, 종류 불일치, 숫자 아닌 값, 0 K 미만	4 / 4가 메시지와 종료 코드 $\neq 0$
의존성	import 목록 확인	표준 라이브러리만

이번에 하지 않는 것은 다음과 같다. 필요해지면 새 목표로 다룬다.

- 온도 차. “ $10\text{ }^{\circ}\text{C}$ 올랐다”를 $^{\circ}\text{F}$ 로 바꾸는 delta 변환(GNU Units와 Pint가 따로 다루는 것[R4, R5]).
- 복합 단위와 다른 종류. km/h, 넓이, 부피, 시간 등.
- 설정 파일과 배포. 사용자 정의 단위, pip 패키지, 대화형 모드.

5 마일스톤 계획

요청대로 마일스톤은 둘이다. 그림 2처럼 M2는 M1의 변환 함수를 감싸기만 하므로 M1이 끝나야 시작한다.

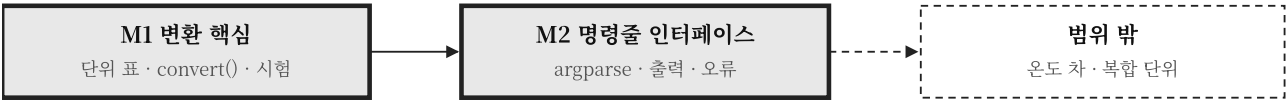


그림 2. 마일스톤 의존 관계. 굵은 테두리가 이번 목표의 두 마일스톤이고, 점선은 이번에 하지 않는 확장이다.

표 5. 마일스톤별 내용과 완료 기준.

마일스톤	내용	완료 기준
M1 변환 핵심	convert.py에 길이·무게 배수 표, 온도 식, convert(value, from, to) 함수. test_convert.py에 unittest 시험.	표 4의 정확성·왕복 일관성 시험 통과. 없는 단위, 종류 불일치, 0 K 미만은 예외 발생.
M2 명령줄 인터페이스	argparse로 값·단위 두 개를 받아 결과 출력 (유효숫자 10자리). 잘못된 입력은 한 줄 메시지와 종료 코드 2.	python convert.py 100 km mi → 62.13711922 mi. 표 4의 오류 4종 시험 통과. 음수 입력(-40 C F) 처리.

6 위험과 대응

작은 도구라 위험도 작지만, 표 6의 네 가지는 시험으로 미리 막는다.

표 6. 위험과 대응.

위험	가능성 / 영향	대응
부동소수점 오차	높음 / 낮음. 예: 0.1 + 0.2 = 0.30000000000000004	출력은 유효숫자 10자리로 반올림. 시험은 같은 값 비교 대신 상대 오차 1e-12로 비교.
음수가 옵션으로 해석됨	중간 / 중간. 3.11에서 -1e3은 위치 인자로 인정되지 않는다[R6].	옵션을 음수처럼 생긴 이름으로 만들지 않는다. -40 시험 포함, 지수 표기 음수는 -- 사용을 오류 메시지와 도움말에 안내.
단위 기호 모호성	낮음 / 낮음. c와 c, t(톤)의 뜻	설계 문서에서 대소문자 규칙과 별칭을 정한다. 없는 단위 오류는 쓸 수 있는 단위 목록을 함께 보여 준다.
잘못된 계수 입력	낮음 / 높음. 결과가 조용히 틀린다.	계수를 표 3의 정의값 그대로 적고, 시험이 정의값을 따로 다시 확인한다.

참고 출처

[R1] NIST. NIST Guide to the SI, Appendix B.8: Factors for Units Listed Alphabetically (SP 811). <https://www.nist.gov/pml/special-publication-811/nist-guide-si-appendix-b-conversion-factors/nist-guide-si-appendix-b8> (2026.10.09 확인)

[R2] NIST. NIST Guide to the SI, Appendix B.9: Factors for Units Listed by Kind of Quantity or Field of Science (SP 811). <https://www.nist.gov/pml/special-publication-811/nist-guide-si-appendix-b->

conversion-factors/nist-guide-si-appendix-b9 (2026.10.09 확인)

[R3] Wikipedia. International yard and pound.

https://en.wikipedia.org/wiki/International_yard_and_pound (2차 출처, 1959년 협정 원문은 확인 안 함)

[R4] Pint documentation. Temperature conversion.

<https://pint.readthedocs.io/en/stable/user/nonmult.html>

[R5] GNU Units. <https://www.gnu.org/software/units/>

[R6] Python Software Foundation. argparse — Parser for command-line options, arguments and subcommands, “Arguments containing -”. <https://docs.python.org/3/library/argparse.html>

[D1] 저장소 초기 커밋 254d145 “init” (파일 없음), 2026.10.09 확인.

[D2] 실행 환경 확인: `python --version` → Python 3.11.16, 2026.10.09.